

The Definitive Guide to RAG Architectures and Implementation

A Comprehensive Chapter-by-Chapter Technical Reference Manual

CHAPTER 1: Foundations of Retrieval-Augmented Generation (RAG)

1.1 The Paradigm Shift: Closed-Book to Open-Book

Traditional Large Language Models (LLMs) function primarily as **closed-book systems**, relying entirely on the static parametric knowledge encoded within their weights during pretraining 1, 2. While powerful, this design introduces fundamental bottlenecks:

- * **Knowledge**

Staleness: Training is computationally bounded by a strict cutoff date, leaving the model unaware of events succeeding that window 1, 3.

- * **Domain**

Blindness: LLMs lack visibility into private enterprise databases, proprietary codebases, or dynamically updating workflows 1, 3.

- *

Confabulation (Hallucination): Under high semantic uncertainty, the model's auto-regressive decoding objective optimizes for linguistic plausibility over factual correctness, yielding confident but fabricated statements 2, 3.

Formalised in 2020 by **Patrick Lewis et al.** at **Facebook AI Research (FAIR)**,

Retrieval-Augmented Generation (RAG)

introduces an **open-book architecture** 2, 4. The system decouples the **reasoning engine** (the LLM) from the **knowledge source** (the external corpus) 1, 4. Rather than forcing the model parameters to store vast factual records, the generator is provided with a curated, query-relevant context window, transitioning the task from parametric memorisation to in-context synthesis 1, 4. ##### 1.2 Strategic Advantages in

Production RAG provides immediate, production-ready mitigation for core LLM limitations without the massive compute overhead of parameter fine-tuning 3, 5: 1. **Freshness**

(Real-time updates): Updating the external document repository automatically exposes new facts to the system without requiring retraining 1,

3. 2. **Verifiability & Auditability**: Generating answers strictly grounded in retrieved documents allows for **deterministic provenance tracking** and explicit, source-attributed citation injection 2,

6. 3. **Access Control and Permissions**: Access policies and data security models can be directly enforced at the retrieval layer, restricting document access per user without modifying the model 6, 7.

1.3 Architectural Primitives of the Linear

Pipeline A standard RAG pipeline operates over two distinct phases: **Ingestion (Offline)** and **Query/Inference (Online)** 8, 9.

=====

=====

===== INGESTION PIPELINE

(OFFLINE)

=====

=====

===== [Raw Data] → [Data Extraction] →

[Chunking] → [Vectorisation] → [Indexation] →

[Vector DB] (PDF/HTML) (Text Parsing) (Size,

Overlap) (Embedding) (HNSW/IVF)

(Chroma/Milvus)

=====

=====

===== QUERY PIPELINE (ONLINE)

=====

=====

===== [User Query q] | ▼ [Query

Encoder] | (Dense Vector) ▼ [Vector Database]

→ (ANN Cosine Match) → [Top-k Chunks R] | |

1. **Introduction**

▼ [Prompt Augmentation

Layer] | (Injected Prompt) ▼ [LLM Generator] →

[Grounded Answer a] ##### 1.3.1 Document

Ingestion & Text Extraction Raw, heterogeneous data structures (PDFs, HTML, markdown, spreadsheets) are ingested and converted into clean plain-text streams 10, 11. Visual or structural metadata (such as page numbers, tables, and document hierarchies) are extracted and preserved as metadata fields linked to each text segment 10, 11. ##### 1.3.2 Text Chunking

(Mathematical & Structural Constraints) Long, continuous documents cannot be passed directly to vector embedding models or LLMs due to model input dimensions and context limits 10, 12.

Documents are therefore partitioned into smaller segments, known as **chunks** 10, 12. * **Chunk Size**

(\$C_s\$): The maximum length of a segment (typically defined in tokens, e.g., 256 to 1024 tokens) 10. A value of \$C_s\$ that is too small leads to context fragmentation, while a value of \$C_s\$ that is too large introduces excessive semantic noise 10. * **Chunk Overlap (\$C_o\$)**:

The sliding window overlap between adjacent chunks (typically 10–20% of \$C_s\$) 10. This sliding window mathematically prevents semantic truncation and the loss of facts at arbitrary boundary cuts 10. ##### 1.3.3 Embedding

Generation (Vectorisation) Let a chunk \$c_i\$

represent a clean sequence of tokens. An **embedding model** Ψ maps the sequence c_i to a high-dimensional vector space: $\mathbf{e}_i = \Psi(c_i) \in \mathbb{R}^d$ where d typically spans 384, 1024, or 3072 dimensions depending on the dense retriever model used 12, 13. Similarity between two text blocks is computed as the geometric distance (most commonly **cosine similarity**) between their vector representations

$$\text{sim}(c_i, c_j) = \frac{\mathbf{e}_i \cdot \mathbf{e}_j}{\|\mathbf{e}_i\| \|\mathbf{e}_j\|} \quad \text{#####}$$

1.3.4 Vector Indexing & Storage

Vector databases index the high-dimensional embeddings using Approximate Nearest Neighbor (ANN) search structures such as **Hierarchical Navigable Small World (HNSW)** or **Inverted File with Product Quantization (IVF-PQ)** 14, 15. This indexing strategy decouples the retrieval latency from the linear scaling of the document corpus, enabling sub-millisecond retrieval across millions of dense vectors 14, 16. #####

1.3.5 Runtime Prompt Augmentation & Generation

At query time, the user query q is mapped to a vector $\mathbf{e}_q = \Psi(q)$ 11. The vector database returns the k most similar chunks $R = \{c_1, c_2, \dots, c_k\}$ 11. A prompt template, known as the **Prompt**

Augmentation Layer, merges the retrieved context with the user query into a single structured string $P\{\text{aug}\}$ 8, 11:

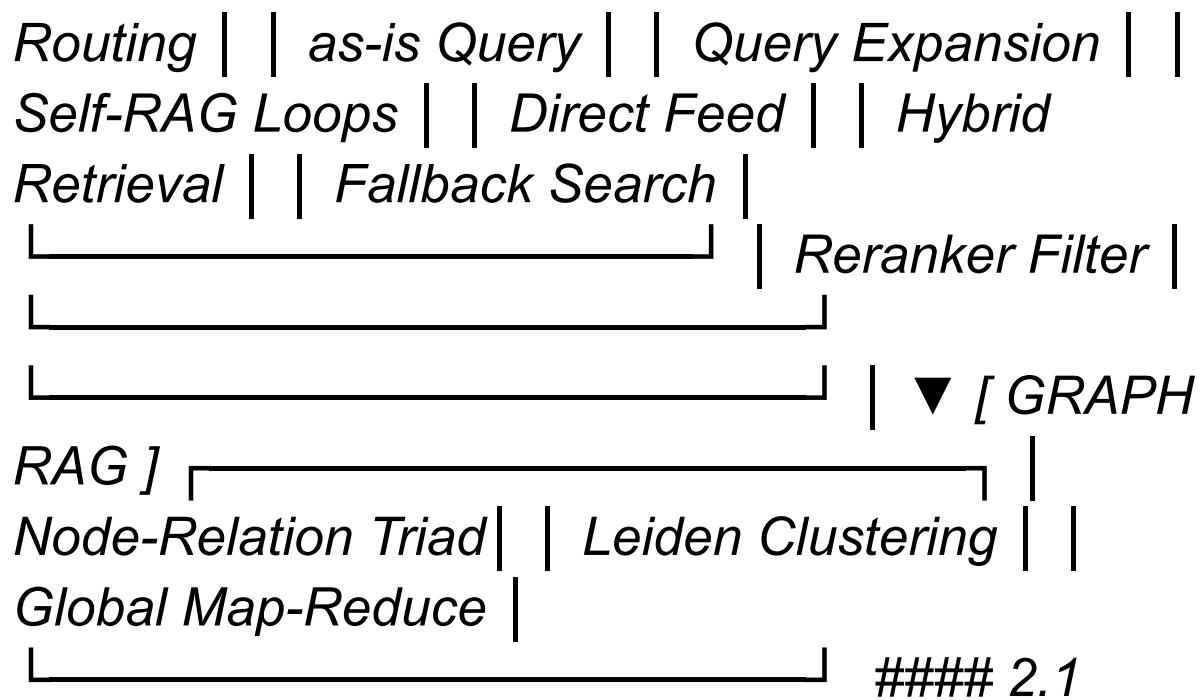
$\text{Prompt}\{\text{Template}\} = \text{"Context: } R \text{ \n\n Question: } q \text{ \n\n Answer only based on context."}$ The LLM is prompted with

this context-grounded payload to generate the final, factually grounded answer a 8, 11. --- ###
CHAPTER 2: The Evolutionary Generations (Naive → Advanced → Modular → Graph) RAG has progressed rapidly from basic similarity pipelines to highly complex, structured multi-agent systems 6, 17.

|| THE
 RAG EVOLUTIONARY ROADMAP ||

[
 NAIVE RAG] → [
 ADVANCED RAG] → [
 MODULAR RAG]

Vector
 Search | Semantic Chunking | Agentic



Naive RAG (First Gen, ~Early 2023) Naive RAG implements a straightforward "chunk, embed, retrieve, generate" workflow 18, 19. The search relies on a single vector search step using the raw user query, returning flat, unstructured text fragments to the prompt context 6, 18. #####

Systemic Bottlenecks: * **Vocabulary Mismatch:** Dense embeddings fail on exact keywords, identifiers (e.g., error codes like `ERR_404`), or domain-specific nomenclature 14, 20. *

Lost-in-the-Middle Phenomenon: Transformer attention weights exhibit a high positional bias, frequently missing key facts located in the middle of long, concatenated prompt contexts 21, 22. *

Context Fragmentation: Arbitrary structural slicing splits a single, unified argument across

multiple chunks, degrading the model's ability to track long-range logical reasoning 22, 23. ##### 2.2

Advanced RAG (Second Gen, 2023–2024)

Advanced RAG addresses Naive RAG's limitations by adding smart optimization blocks across three key stages 19, 24: ##### 2.2.1 Pre-Retrieval

Optimization * **Semantic Chunking**: Instead of hard character or token limits, chunks are dynamically split by tracking similarity changes between sequential sentences, preserving

cohesive semantic boundaries 19, 24. * **Query**

Expansion & Reformulation: A lightweight model generates N semantically equivalent but lexically diverse variations of the query $Q' =$

$\{q_1, q_2, \dots, q_N\}$ 25, 26. This strategy expands the search space to capture temporal synonyms, alternate spellings, and cross-lingual matches 26. * **Hypothetical Document**

Embeddings (HyDE): The LLM is prompted to generate a speculative, hallucinated answer to the query 14, 25. This hypothetical text is then embedded to query the database 14. This method exploits the property that answer-to-answer vector matching yields a higher cosine similarity than question-to-answer matching 14. ##### 2.2.2

During-Retrieval (Hybrid Sparse-Dense Search) To

resolve exact nomenclature search failures, systems implement **hybrid search**, combining dense semantic vectors with sparse lexical keyword matching (such as **BM25**) 9, 14: * **BM25**: Scoring is based on term frequency (\$TF\$) and inverse document frequency (\$IDF\$), capturing precise matching strings and specific product codes 14. * **Dense Vectors**: Dense embeddings capture the broader semantic intent, synonyms, and conceptual structures 14. To combine these disjointed rank lists, **Reciprocal Rank Fusion (RRF)** aggregates the scores of each document \$d\$ 9, 14:
$$RRF(d) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$
 where \$r_m(d)\$ represents the rank of document \$d\$ in the retrieved list from retriever \$m\$, and \$k\$ is a smoothing constant (typically set to 60) 14. ##### 2.2.3 Post-Retrieval Optimization * **Cross-Encoder Reranking**: First-pass retrieval uses fast, independent bi-encoder vector matches 14. In the post-retrieval stage, a computationally intensive **Cross-Encoder Reranker** processes the query and the top candidates together to capture fine-grained token-level interactions, sorting the most relevant documents to the extreme edges of the context window to mitigate attention drift 14, 21. *

Contextual Compression: Irrelevant text spans are dynamically pruned from the retrieved chunks, returning only the most informative sentences to the LLM to optimize the context budget 21. #####

2.3 Modular RAG (Generation 2.5) Modular RAG transitions the system from a static, linear pipeline into a dynamic network of swappable, agentic components 27, 28. A central routing model evaluates the incoming query complexity and adaptively determines the execution workflow 27: *

Self-RAG: The system learns to critique its own generations by outputting special reflection tokens, determining whether external retrieval is needed and assessing generation quality on the fly 29, 30.

* **Corrective RAG (CRAG):** An evaluation module inspects the retrieved documents 27, 31. If the returned chunks are deemed irrelevant, the system triggers a fallback search (e.g., querying the web or public search APIs) to pull in correct context 27, 31. #####

2.4 Graph RAG (Third Gen, Relation-Centric Evolution) Standard vector databases struggle with multi-hop reasoning (e.g., "Where did Project Alpha's manager work previously?") because they cannot traverse structural relationships across isolated text chunks 32, 33. Graph RAG models the document corpus

as an entity-relation graph to support structured relational reasoning 32, 34. ##### 2.4.1 Ingestion Phase: Entity-Relation Extraction The system parses unstructured documents to extract named entities (subjects/objects) and their relationships, mapping them to **subject-predicate-object triples** 35, 36: $\langle \text{Subject}, \text{Relation}, \text{Object} \rangle$ These extracted entities are aligned using vector similarity-based **Entity Resolution** to merge synonyms (e.g., consolidating "J. Smith" and "John Smith" into a single node) 37, 38. ##### 2.4.2 Community Clustering (Microsoft's Leiden Paradigm) For large-scale, global document understanding (e.g., "What are the main risks identified in the financial statements?"), standard local entity traversal is insufficient 37, 39. Microsoft GraphRAG introduces a hierarchical community summarisation workflow 37: 1. **Clustering**: The global graph is partitioned into densely connected groups (communities) using the **Leiden Algorithm** 37. 2. **Summarisation**: An LLM generates a comprehensive summary for each community, consolidating its nodes and relationships into a structured text document 37. 3. **Global Search**: Global queries are mapped across all community

summaries using a **map-reduce pattern**, aggregating individual community insights into a synthesized global answer 37, 39. ##### 2.4.3 Dual-Level Graph Retrieval (LightRAG) LightRAG optimizes Graph RAG by indexing entities and relationships at two distinct granularities 40: *

Low-Level Entities: Captures specific, local facts and precise entity details 40. * **High-Level**

Keywords: Captures broader, conceptual themes across the corpus 40. At query time, the system combines both levels to retrieve relevant subgraphs alongside textual chunks, balancing local accuracy with global context 40. --- ###

CHAPTER 3: Advanced Long-Document & Memory RAG Architectures (2025–2026) Recent architectural breakthroughs focus on preserving global coherence, tracking dynamic relational structures, and maintaining logical consistency across extremely long contexts 41, 42. ##### 3.1 Mindscape-Aware RAG (MiA-RAG) Vanilla RAG lacks global context, treating individual chunks as isolated snippets 43, 44. **MiA-RAG** addresses this by constructing a hierarchical, abstract representation of the entire document, termed the **Mindscape (\$S\$)**, which emulates human global semantic memory 43-45.

===== MiA-RAG SYSTEM FLOW =====

===== [Long Document] → [Hierarchical Summarisation] → [Global Mindscape S] | ▼
 Query q → [Residual Query Encoder] → Enriched Query $\tilde{q}$ | ▼ [MiA-Emb Retriever] →
 [Context Chunks] | ▼ [MiA-Gen Generator] →
 Grounded Answer #####

3.1.1 Hierarchical
 Mindscape Construction The Mindscape S is
 built in a bottom-up, hierarchical manner 46: 1.

Chunk Summarisation: A summarizer model
 generates a concise summary s_i for each raw
 text chunk c_i : $s_i =$

$M_s(\text{INST}\{\text{sum}_c\}, c_i)$ 2. **Global**

Consolidation: The sequence of chunk
 summaries is concatenated and recursively
 summarized to construct a single global
 representation S : $S =$

$M_s(\text{INST}\{\text{sum}_g\}, [s_1, s_2, \dots, s_n])$ #####

3.1.2 Dual-Pass Representation

and Training Both the retriever (**MiA-Emb**) and the
 generator (**MiA-Gen**) are trained using this global
 semantic scaffold S to align local evidence with

global context 44, 45: * **MiA-Emb Query**

Encoding & Residual Integration: To inject global context without washing out the user's specific query intent, the system employs

Residual Integration 47-49. Let $Q =$

$[\text{INST}; \text{emb}; q_i; d_q; S; d_t]$ be the query sequence, where d_q is the query end delimiter and d_t is the active task delimiter

(node or chunk mode) 48, 50. The system extracts the hidden states at these delimiters, denoted as $\mathbf{h}_q$ and $\mathbf{h}_t$ respectively 49.

The enriched query representation

$\tilde{\mathbf{q}}_t$ is computed via a residual connection 49: $\tilde{\mathbf{q}}_t = \delta \cdot \mathbf{h}_q + (1 - \delta) \cdot \mathbf{h}_t$

The model is optimized using a joint contrastive

InfoNCE Loss over positive (c^+) and negative (c^-) candidates 47, 48:

$$\mathcal{L}_{\text{MiA-Emb}} = -\log \frac{\exp(\text{sim}(\tilde{\mathbf{q}}_t, \mathbf{c}^+))}{\exp(\text{sim}(\tilde{\mathbf{q}}_t, \mathbf{c}^+)) + \sum_j \exp(\text{sim}(\tilde{\mathbf{q}}_t, \mathbf{c}_j^-))}$$

* **MiA-Gen Auto-Regressive**

Optimization: The generator is fine-tuned to

produce the grounded answer y_i by conditioning on both the Mindscape summaries and the retrieved chunks 48, 51:

$$\mathcal{L}_{\text{MiA-Gen}} = -\sum_i \sum_{t=1}^{|y_i|} \log P(\theta(y_{i,t} | y_i, \tau_{\text{rag}}))$$
, the prefetching pipeline is

triggered 100. 2. **Sufficiency Classifier**: If current cached documents are sufficient, retrieval is bypassed 100. 3. **Clarity Score**: If current context is incomplete or vague, retrieval is delayed until the context stabilizes 100. 4. **Query Generation**: Formulates and issues the prefetch query, caching the results to be instantly available when token generation hits the reasoning boundary 97, 100.

4.4 Corpus-Grounded Verification

(QuCo-RAG) Dynamic retrieval systems frequently rely on model-internal confidence signals (like token entropy or attention weights) to trigger retrieval 101, 102. However, LLMs can be confidently incorrect, hallucinating entities with low entropy 101, 102. **QuCo-RAG** replaces these internal heuristics with objective, **corpus-grounded statistics** 103, 104.

=====
=====
===== QuCo-RAG SYSTEM PIPELINE

=====

=====

===== [Generated Statement] → [Triplet Extraction] → [Infini-gram Engine] → (Frequency = 0) | ▼ [Trigger Retrieval]

1. **Fact Extraction:** The system parses each generated statement s_t into factual triples using a lightweight extractor

103, 105: $\langle \text{Head Entity}, \text{Relation}, \text{Tail Entity} \rangle$

2. **Corpus Querying (Infini-gram Engine):** The extracted entities are queried against an index of a trillion-token pretraining corpus (using **Infini-gram** structure for instant lookup)

103, 104: * **Entity Frequency:** Measures how often an entity appears in the pretraining corpus

103, 104. * **Entity Co-occurrence:** Evaluates the co-occurrence frequency of the head and tail entities

103, 104. 3. **Adaptive Triggering:** If the co-occurrence frequency is zero (or falls below a strict threshold τ_{cooc}), the statement is flagged as highly uncertain, triggering external retrieval to verify the fact and self-correct the generation

101, 103. --- ### CHAPTER 5: Specialised and Cross-Modal RAG Systems RAG architectures are increasingly customized to handle complex, heterogeneous data structures, including

structured tables, visually rich document layouts,
and long video streams 42, 106. ##### 5.1

FT-RAG: Fine-Grained Table Reasoning Standard
RAG struggles with structured tabular data
because character-level chunking collapses cell
dependencies and destroys row-column semantic
*alignments 107, 108. **FT-RAG** addresses this by*
decomposing tables into atomic, schema-aware
relational elements 108, 109.

=====
=====
===== *FT-RAG DATA FLOW* =====
=====

===== *[Raw Table] → [Cell-Group*
Decomposition] → [SAT Graph
(Subject-Attribute-Temporal) | ▼ [Dual-Path
Traversal] - Anchor-Driven (Top-Down) -
Attribute-Driven (Bottom-Up) | ▼ [Text-Bridged
Augmentation] ##### 5.1.1 *Entry-Level Table*

*Decomposition Tables are parsed into **Cell***
***Groups** $e = (v, c, e, t, s, k, m)$ 109, 110: * v :*
*The raw cell value 110. * c, e, t, s, k :*
Dimensional metadata, tracking column context,
entity, temporal validity, subject, and key attributes
*110. * m : Provenance coordinates linking the*

cell back to its source table and surrounding document text 110, 111. ##### 5.1.2

Schema-Aware SAT-Graph Construction The cell groups are organized into a hierarchical

Subject-Attribute-Temporal (SAT) graph $G = (V, E)$ 112, 113: * **Subject Sub-tree (T_S)**:

Encodes the structural entity lineage (e.g., nesting "Laptop" under "Hardware") 113, 114. * **Temporal**

Sub-tree (T_T): Normalizes and chronologically structures time intervals 114, 115. * **Attribute**

Node Set (V_A): Serves as the central semantic intersection layer, bridging subject and temporal paths to specific metric definitions 115, 116. *

Value Leaves (V_L): The terminal nodes storing cell values, anchored by unique path coordinates $T_S \times T_T \times V_A$ 117.

5.1.3 Dual-Path Traversal & Text-Bridged

Augmentation At query time, the system performs a topology-aware search using two complementary traversal paths 118: * **Anchor-Driven Forward**

Traversal: Traverses top-down from subject and temporal anchors to resolve matching attribute values 118, 119. * **Attribute-Driven Reverse**

Traversal: Back-traces bottom-up from specified attributes to identify relevant subject and temporal contexts 119, 120. To preserve context, the

retrieved cell values are merged with their surrounding document text to generate a unified, factually complete prompt context 121, 122. #####

5.2 MG²-RAG: Multi-Granularity Multimodal Graph Many multimodal document layouts contain figures and flowcharts that cannot be fully captured by text-only translation pipelines 123, 124.

MG²-RAG constructs a hierarchical multimodal knowledge graph to align text and visual regions without informational loss 123, 124.

=====

=====

===== MG²-RAG HYBRID INDEX

=====

=====

===== [Text Chunks] → [spaCy NER / POS] —┐ └→ [Unified Multimodal Nodes]

[Page Images] → [SAM3 Segment] —┐ (Fuses Text and Visuals) | ▼ [Personalized PageRank]

→ Top-K ##### 5.2.1 Graph Construction &

Modality-Preserving Fusion 1. **Textual Structure**

Extraction: Extracts entities and parses grammatical relations using rule-based dependency parsing over verbs and nominal modifiers, avoiding expensive LLM calls 125, 126.

2. **Entity-Driven Visual Grounding:** Rather than

*parsing the entire image, the system uses the extracted entity names as semantic prompts in an open-vocabulary segmentation model (**SAM3**) to localize and crop entity-relevant image regions*

*126. 3. **Modality Fusion**: Aligns textual entities and grounded visual regions into unified, atomic multimodal nodes, preserving fine-grained details across modalities 124, 125. 4. **Heterogeneous***

Edge Integration**: Connects nodes across three dimensions: **Contextual Edges** (preserving document layout hierarchy), **Semantic Edges

*(encoding grammatical dependencies), and **Grounding Edges** (aligning cropped objects with their source images) 127, 128. ##### 5.2.2*

Multi-Granularity Graph Propagation Query

features are encoded into a shared vector space

129, 130. Initial similarity matching activates a set

*of **seed nodes** across the graph 130, 131. The*

system then propagates these activations using

***Personalized PageRank (PPR)**, traversing the*

heterogeneous edge topology to resolve multi-hop,

cross-modal queries 131. ##### 5.3 TV-RAG:

*Temporal Video RAG **TV-RAG** focuses on long*

video retrieval and understanding by aligning

*visual, audio, and subtitle streams 31, 132. **

***Temporal Offset Ingestion**: Video files are parsed*

and indexed alongside precise temporal timestamps 31. * **Semantic Entropy-Weighted Sampling:** To prevent context window saturation, frames are sampled based on semantic entropy, prioritizing highly informative transitions while pruning redundant, static scenes 31. ##### 5.4

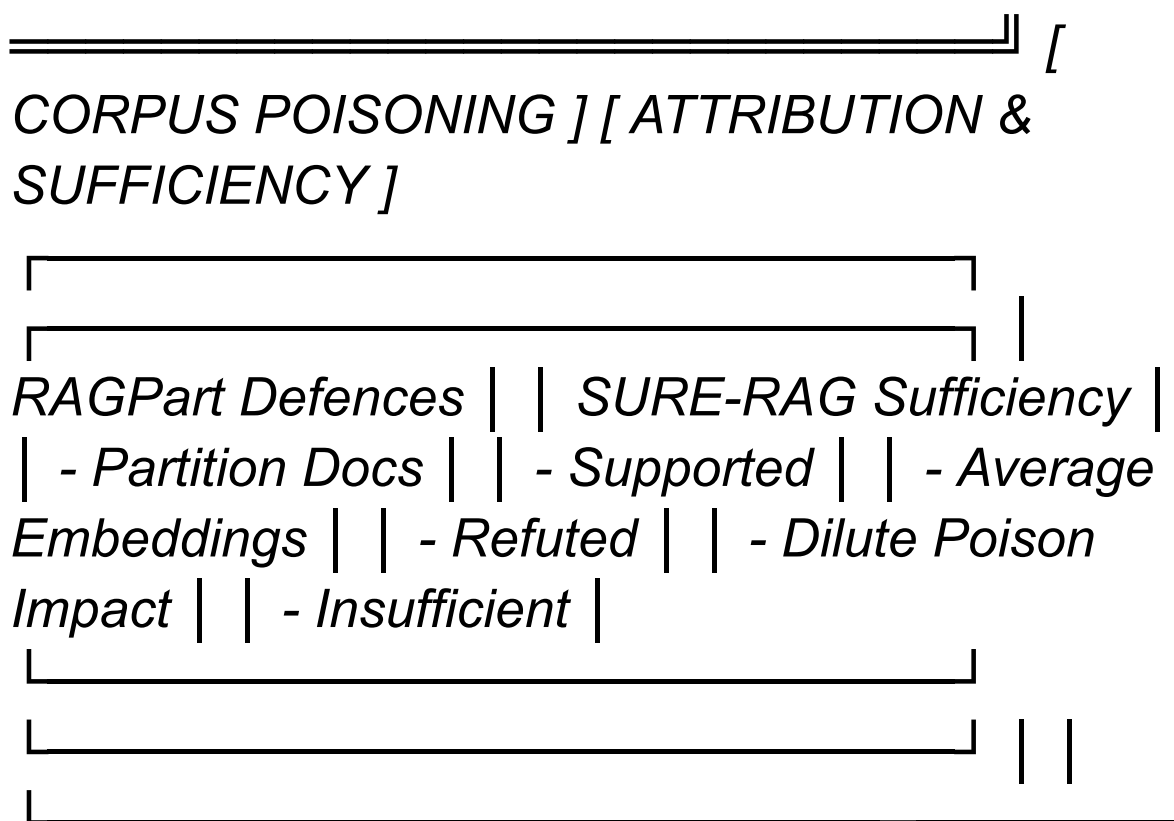
Historical Multilingual OCR-degraded RAG

Designed for noisy historical newspapers and archival documents 133, 134: * **OCR Error**

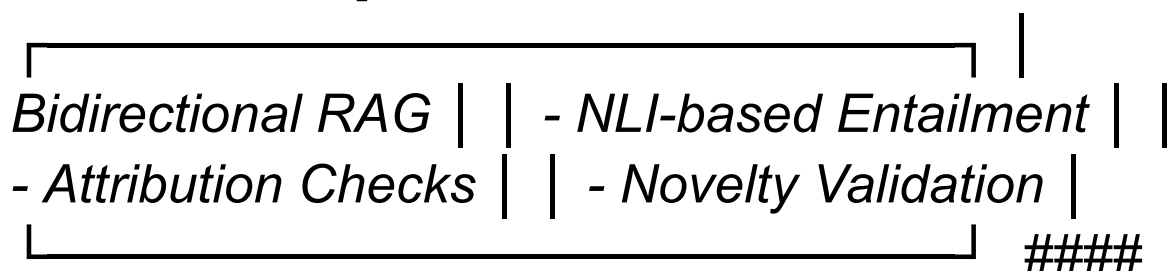
Mitigation: Integrates fuzzy search and wild-card lexical matching with dense embeddings to handle OCR characters corruptions 133. * **Temporal**

Language Drift: Evaluates semantic transitions over time, using query expansion templates to translate modern search terms into archaic synonyms 26, 135. --- ### CHAPTER 6: RAG

Security, Safety, and Verification Deploying RAG in production introduces critical security vulnerabilities, including adversarial corpus poisoning and the propagation of early reasoning errors 136, 137.



▼ [CONTROLLED WRITE-BACK]



6.1 Corpus Poisoning Defenses (RAGPart & RAGMask) Adversaries can inject maliciously crafted documents into the retrieval corpus to manipulate LLM generation, a vulnerability known as **corpus poisoning** 137, 138. * **RAGPart**: Dilutes the impact of poisoned segments 139. The system partitions each document into \$N\$

fragments, generates embeddings for all $\binom{N}{k}$ combinations of size k , and averages their vector representations 139, 140. This leverages the dense retriever's semantic alignment to isolate and suppress the influence of poisoned fragments during vector matching 139, 141. * **RAGMask**: Detects suspicious tokens at the retrieval stage 142. The system masks tokens within the retrieved documents and tracks the resulting shift in query similarity 142. An abnormal similarity shift flags potential malicious injections, allowing the system to sanitize the document before it reaches the generator 142. ##### 6.2 SURE-RAG: Evidence Sufficiency Verification Standard RAG verifiers focus on basic semantic relevance, which fails to detect cases where a document is relevant but fails to justify the answer 143, 144. **SURE-RAG** acts as a validation layer that evaluates **evidence sufficiency** before answering 143, 145. ##### 6.2.1 Three-Way Verification Taxonomy SURE-RAG classifies each candidate answer against the retrieved evidence into three distinct categories 144, 146: 1. **SUPPORTED**: The evidence completely entails the assertions in the answer 144, 146. 2. **REFUTED**: The evidence explicitly contradicts the

assertions 144, 146. 3. **INSUFFICIENT**: The evidence is relevant but fails to justify the answer (e.g., missing reasoning steps or incomplete facts) 144, 146. The system only generates an answer if the evidence is classified as **SUPPORTED**, safely abstaining otherwise 146, 147. ##### 6.2.2

Set-Level Sufficiency Aggregation Because evidence sufficiency is a set-level property, simple average-pooling over isolated passages can dilute refutations or mask missing reasoning links 145.

SURE-RAG maps all pair-level claim-evidence distributions into five structured feature blocks 145,

147: * **Coverage**: The fraction of assertions supported, refuted, or insufficient 148. * **Relation**

Strength: The maximum support and refutation scores across all passages 149. * **Uncertainty**:

The entropy, disagreement, and conflict levels across the retrieved set 149. The system

processes these features using a trained aggregator model to produce a calibrated

selective-answering decision 147, 150. ##### 6.3

Bidirectional RAG (Self-Improving Write-Back

Loops) Standard RAG operates as a static, read-only system, discarding valuable

model-generated insights (such as syntheses or

summaries) 151, 152. **Bidirectional RAG** enables

safe, dynamic corpus expansion by writing high-quality generated responses back to the retrieval database 136, 151. ##### 6.3.1

Multi-Stage Acceptance Layer To prevent the propagation of hallucinations, every candidate write-back document must pass three validation steps 136:

1. **Grounding Verification (NLI Entailment)**: Evaluates the Natural Language Inference (NLI) entailment probability for each sentence against the retrieved source documents to ensure strict factual grounding 153. 2.

Attribution Checking: Verifies that all generated claims are directly attributable to the source documents 136, 151. 3. **Novelty Detection**:

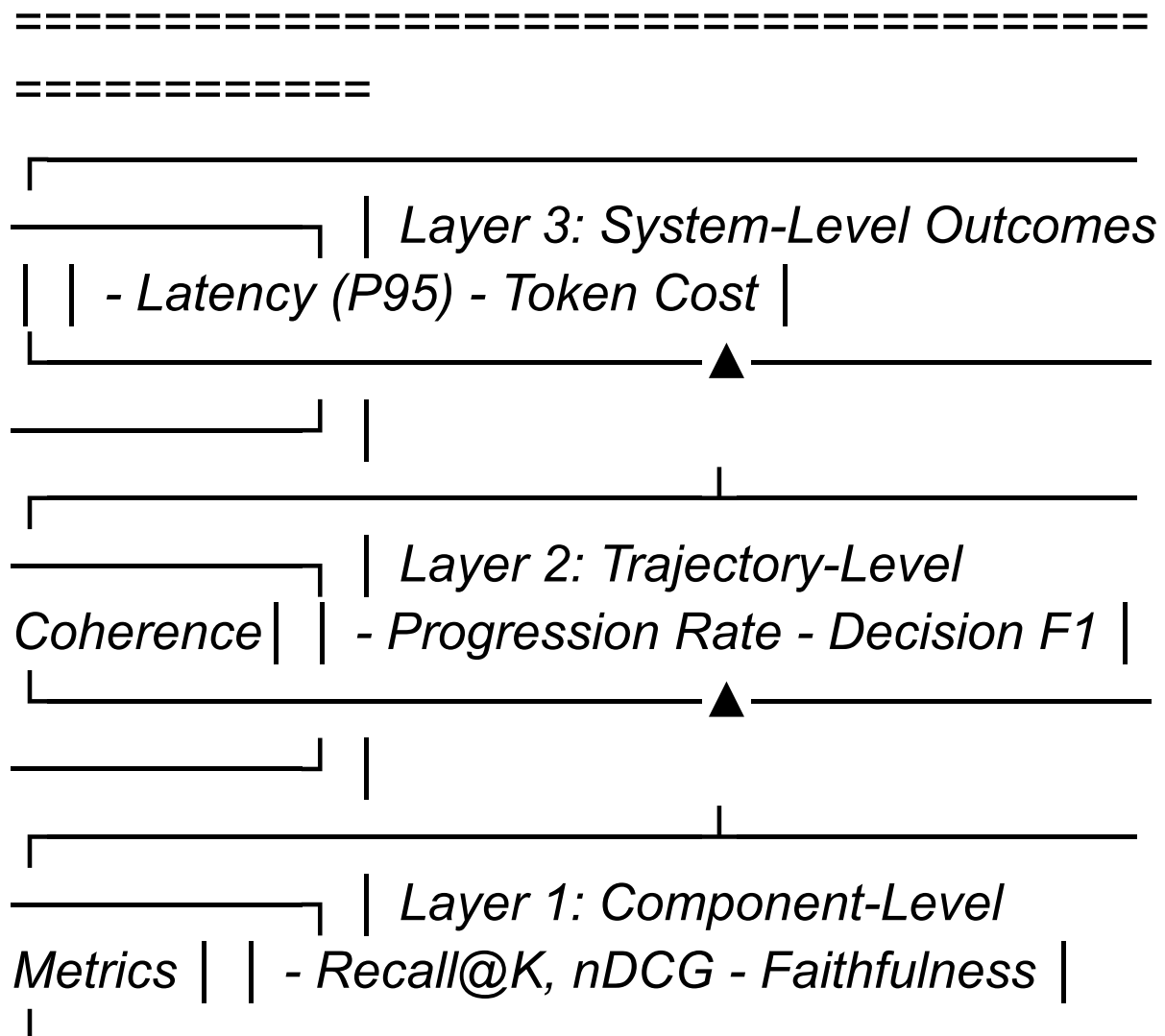
Prevents redundancy by checking that the generated response introduces new, synthesised insights not already present in the corpus 136, 151. --- ###

CHAPTER 7: Evaluation, Metrics, and Production Deployment A production-ready RAG application requires separating **retrieval quality** and **generation quality** into distinct evaluation dimensions 154.

=====

=====

===== **LAYERED RAG EVALUATION** =====



7.1 Quantitative Metric

Formulation * **Recall@K**: Measures whether the target document is present in the top-K retrieved results 154. * **nDCG@K (Normalized Discounted Cumulative Gain)**: Evaluates retrieval quality by scoring document relevance weighted by its position in the rank list 154: $DCG@K = \sum_{i=1}^K \frac{rel_i}{\log_2(i + 1)}$ * **MRR (Mean Reciprocal Rank)**: The average of the reciprocal ranks of the first correct document 154:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Faithfulness (Hallucination Metric): Measures the fraction of generated claims that are directly supported by the retrieved context, evaluated using NLI or LLM-as-a-judge patterns 155, 156. * **Answer Relevance:** Evaluates whether the generated answer directly addresses the user query 156.

7.2 Production Orchestrators & SDKs 1.

LangGraph: Specifically designed for stateful, cyclic multi-agent systems, providing fine-grained control over state preservation and reflection loops 157, 158. 2. **Google ADK (Agent Development Kit):** Provides structured, hierarchical routing loops, utilizing the Model Context Protocol (MCP) to standardize external tool interfaces 158, 159. 3.

AutoGen: An event-driven framework optimized for multi-agent conversations and collaborative task execution 160, 161. 4. **CrewAI:** A process-driven framework focused on sequential task execution and role-based division of labour 160, 161. --- #### CHAPTER 8: Alternative Paradigms — RAG vs. LLM Wiki RAG operates on a static document index, re-deriving knowledge and resolving reasoning steps from scratch for every query 23, 162.

RAG vs. LLM WIKI

[RAG] —(Query-Time Traversal)—→ [Flat Chunks] → [Re-derive on Query] | (System Contrast) ▼ [LLM WIKI]
—(Offline Compiler)—→ [Evolving Markdown] → [Compound Knowledge] (Wikilinks / Git)

8.1 The Re-derivation Problem In traditional RAG pipelines, intermediate insights, synthesised connections, and resolved contradictions are lost as soon as the chat session ends 23, 162. The system is mathematically incapable of accumulating knowledge over time 162.

8.2 The LLM Wiki Paradigm **LLM Wiki** treats the language model not as a passive retriever, but as an active **wiki editor** 163. Factual segments are compiled into human-readable Markdown files, dynamically maintained and recursively updated by the model as new documents are ingested 163. ##### 8.3 Ingestion and Compilation Operations * **Dynamic Intake:** During ingestion, the model evaluates the new document, writes a concise summary page, and

updates relevant entity files 164. It automatically inserts **cross-reference wikilinks** (e.g., [[John Kim]] or [[Project Alpha]]), updating up to 15 files concurrently to preserve relational continuity 164, 165. * **Failsafe Linting**: Periodically, the system triggers a background linter to identify orphan pages, detect factual contradictions across documents, and identify information gaps to guide subsequent web searches 166. * **Version Control**: Because the wiki is stored as flat markdown files, the system can leverage standard git repositories to track history, enable branching, and support collaborative human review 160, 167. ##### 8.4

The Practical Hybrid Endpoint: Wiki-as-a-Source
The ultimate evolution of team knowledge systems combines both paradigms, using the dynamic, **LLM-maintained wiki itself as the retrieval source for downstream RAG** 23, 168. This hybrid architecture merges the real-time speed of RAG with the structured, synthesised depth of an evolving wiki, providing a complete, self-improving enterprise knowledge loop 23, 168. --- ###

SUMMARY COMPARISON: RAG

ARCHITECTURES | Dimension | Naive RAG 35 |
Advanced RAG 21, 35 | Graph RAG 35, 39 |
Agentic RAG 169, 170 | LLM Wiki 23, 171 || :--- |

:--- | :--- | :--- | :--- | :--- || **Data Format** | Flat text
chunks 35 | Chunks with metadata 35 |
Entity-Relation Triples 35 | Multi-Granularity
Indices 169 | Structured Markdown Files 171 ||
Retrieval Strategy | Cosine Vector Similarity 172 |
Hybrid (Dense/BM25) + Rerank 172 | Topology
Traversal 172 | Dynamic Tool Invocation 169 |
Direct Index Matching 167 || **Relational**
Reasoning| Minimal 35 | Low 35 | High 35 | High
(Multi-step) 169 | Extremely High (Wikilinks) 171 ||
Knowledge State | Static 162 | Static 162 | Static
173 | Dynamic Scratchpad 169 | Compounding /
Evolving 171 || **Latency Profile** | Low 172 |
Medium 172 | High 174 | High (Multi-step) 170 |
Low 23 | --- 🔍 Want to build a custom mock
pipeline in our computing environment to test
these concepts? We can write a Python script
demonstrating a hybrid BM25/dense vector
retriever with Reciprocal Rank Fusion (RRF) on
your own data.